Autoscaling

add capacity when demand rises, remove it when it falls

Phase 3 videos on YouTube

In one line: autoscaling adjusts capacity to demand; target tracking keeps a metric near a target, the right metric depends on the workload (CPU for CPU-bound, requests or concurrency for I/O-bound APIs, queue depth for workers), scheduled and predictive scaling cover the reaction lag, and limits and stabilisation windows stop flapping and runaway bills.

1 Four servers at night, forty at lunch

Your API needs 4 servers at night and 40 at lunchtime. Running 40 all day wastes money; running 4 at lunch takes you down. **Autoscaling** adds and removes capacity automatically as demand changes. It only works for **stateless** tiers (lesson 34) that can add instances freely, and only if it scales on the right signal, fast enough.

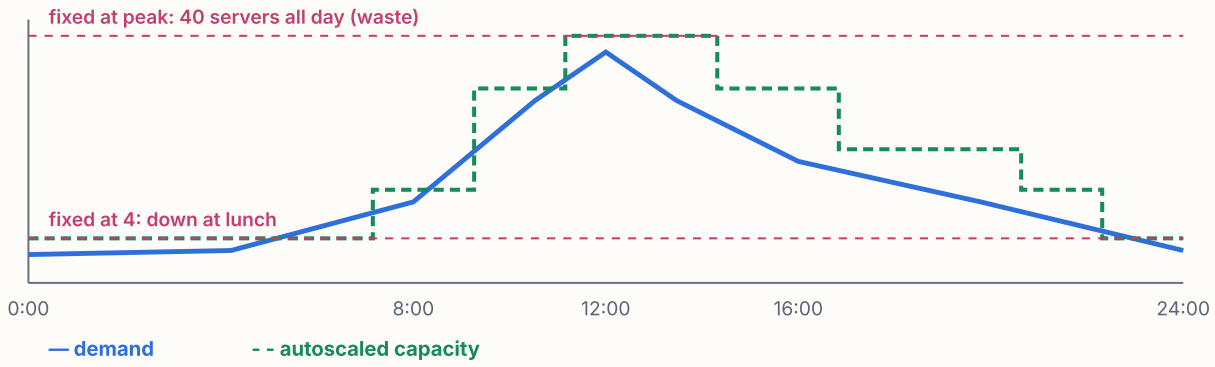


Figure 1. Fixed capacity either wastes money or fails at the peak; autoscaled capacity follows demand in steps.

2 Reactive scaling: target tracking

Target tracking keeps a metric near a target, like average CPU at 60%. If it climbs above, add instances; if it falls below, remove some. The maths is roughly `desired = current × (metric / target)`: 10 instances at 90% CPU with a 60% target → 15 instances. AWS target tracking policies and the Kubernetes **Horizontal Pod Autoscaler** (HPA) both work this way.

3 Which metric?

CPU is a fine default for CPU-bound services. But many APIs are **I/O-bound**: they spend their time waiting on databases and other services. Their CPU stays low even when they're overloaded and latency soars, so a CPU target never triggers. For those, scale on **requests per second per instance**, **concurrent requests**, or latency. For workers, scale on **queue depth** or consumer lag.



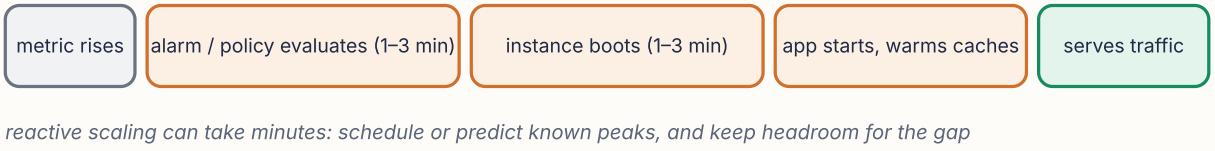
Figure 2. Match the scaling metric to what limits the workload.

4 The lag, and how to beat it

Reactive scaling has lag: the metric must rise, the policy must react, instances must boot and warm up. That can take several minutes, and a sudden spike arrives in seconds.

- **Scheduled scaling** adds capacity before known peaks, like 9:00 every weekday or a planned sale.
- **Predictive scaling** learns daily and weekly patterns and scales ahead of them (AWS predictive scaling).
- **Headroom**: a target of 60% rather than 90% leaves room to absorb a spike while new instances start.

Combine them, and keep reactive scaling as the safety net.



reactive scaling can take minutes: schedule or predict known peaks, and keep headroom for the gap

Figure 3. Why reactive scaling is slow: each step between the spike and new capacity takes time.

5 Stability and limits

- **Cooldowns and stabilisation windows** prevent **flapping** (add, remove, add again). The HPA waits 5 minutes by default before scaling down.
- **Scale up fast, scale down slowly.** Being briefly over-provisioned is cheap; being under-provisioned is an outage.
- **Minimums**, so a quiet night doesn't leave one instance (and no redundancy across zones).
- **Maximums**, so a bug, a retry storm or an attack can't scale your bill to the moon, or overwhelm the database.

WORKED EXAMPLE: CELERY WORKERS WITH KEDA

Celery workers process jobs from a Redis queue. **KEDA** (Kubernetes Event-Driven Autoscaling) watches the queue length, with a target of 20 jobs per worker. 1,000 jobs arrive, so KEDA scales to 1,000 / 20 = **50 workers** (capped by the maximum). When the queue drains, it scales back down, and to **zero** when it's empty, so idle workers cost nothing. The next job waits for a cold start, which is fine for background work.

6 Remember the whole system

- Scaling the app tier sends more load to the **database**, which may not scale at all; 50 workers × 10 connections = 500 connections.
- **Scale to zero** means a cold start for the first request: fine for jobs, often not for user-facing APIs.

- New instances start with cold caches and empty connection pools: add a startup or readiness probe (lesson 7) and warm-up.
- Autoscaling can't fix a bottleneck elsewhere; it can only add more of what you already have.

COMMON MISTAKES

- Scaling an I/O-bound API on CPU, so it never scales while latency explodes.
- No maximum, so a retry storm scales to hundreds of instances and takes the database down.
- Minimum of 1, so there's no redundancy and every scale-up starts from a cold single instance.
- Scaling down too aggressively, then thrashing at every small spike.

7 Key terms

Target tracking: keep a metric near a target value

Scheduled scaling: add capacity at known times

Stabilisation window: wait before acting, to avoid flapping

Scale to zero: no instances while idle; cold start later

HPA: Kubernetes Horizontal Pod Autoscaler

Predictive scaling: forecast demand and scale ahead

KEDA: scales Kubernetes workloads on events like queue length

Headroom: spare capacity for spikes while scaling

8 Interview questions

Q1. Which metric would you autoscale an API on? BASIC

Usually requests per second or concurrency per instance, or latency, rather than CPU alone.

Q2. Why is CPU not always right? BASIC

Because I/O-bound services wait on databases and downstream calls, so they can be overloaded with low CPU; and for queue workers, queue depth is the better signal.

Q3. Traffic jumps 5× at 9:00 every weekday and autoscaling is always late. What do you do? SCENARIO

Add scheduled (or predictive) scaling before 9:00, keep a lower target for more headroom, speed up startup (smaller images, pre-warmed pools), and shed low-priority load during the gap.

Q4. How do you autoscale background workers? SCENARIO

On queue depth or consumer lag, for example with KEDA: desired workers = queue length / target jobs per worker, with min, max and scale-to-zero if cold starts are acceptable.

Q5. After autoscaling to 60 pods, the database falls over. Why? TRICKY

Each pod opens its own connection pool, so connections multiply, and every pod adds queries. Cap the max replicas, use a connection pooler such as PgBouncer, and scale the data tier or cache first.

Q6. Why scale up fast but down slowly? TRICKY

Under-provisioning causes errors and latency for users right away, while over-provisioning for a few minutes only costs a little money; slow scale-down also avoids flapping when traffic is bursty.

Next: SD-F-001 · Why Cache and Where backendroadmap.com